Kathy Wool

Proyecto Final Fullstack Javascript

Integrantes:

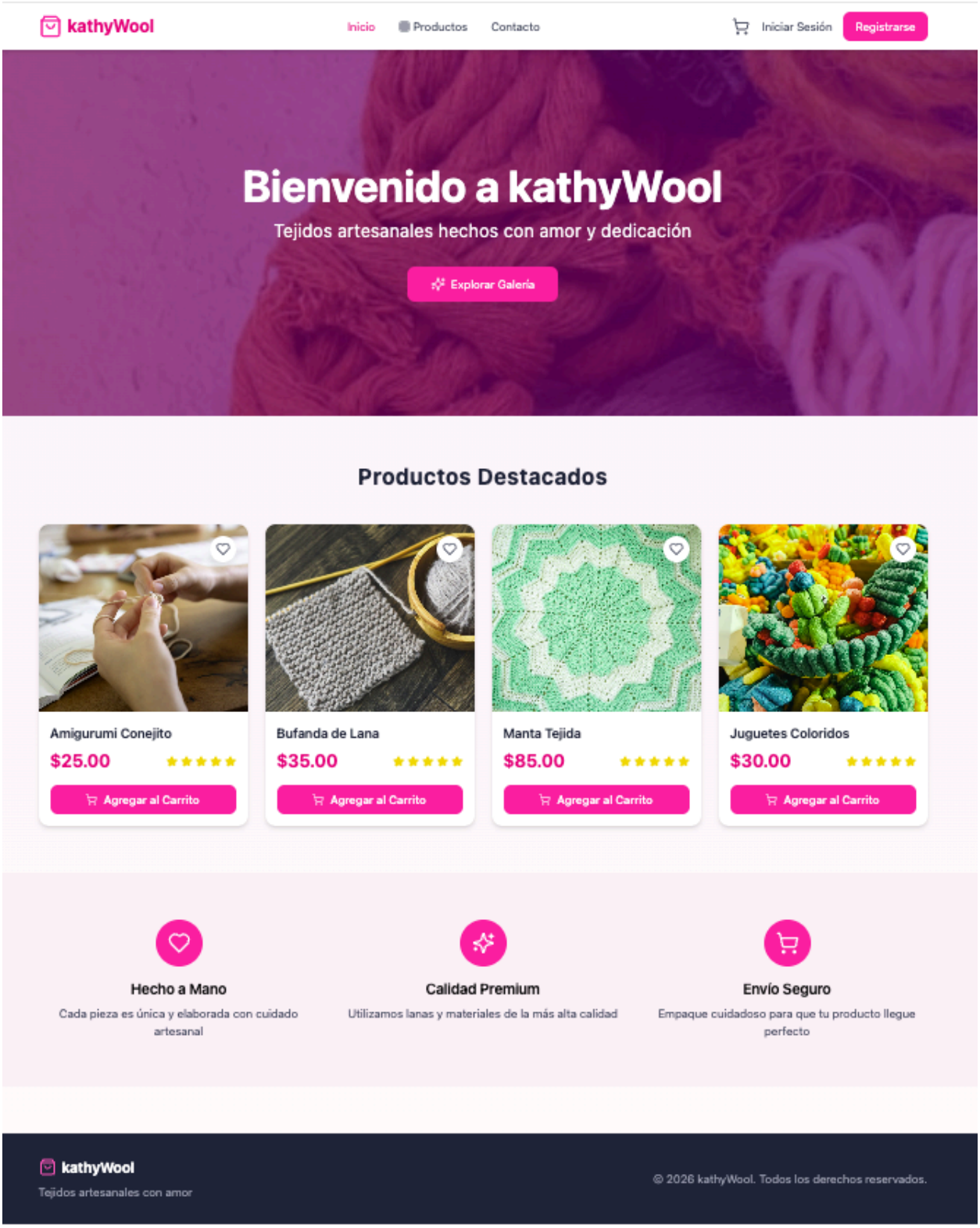
Daniela Elizondo
Katherin Hernández
Sebastián León
Josué Valdivieso

ÍNDICE

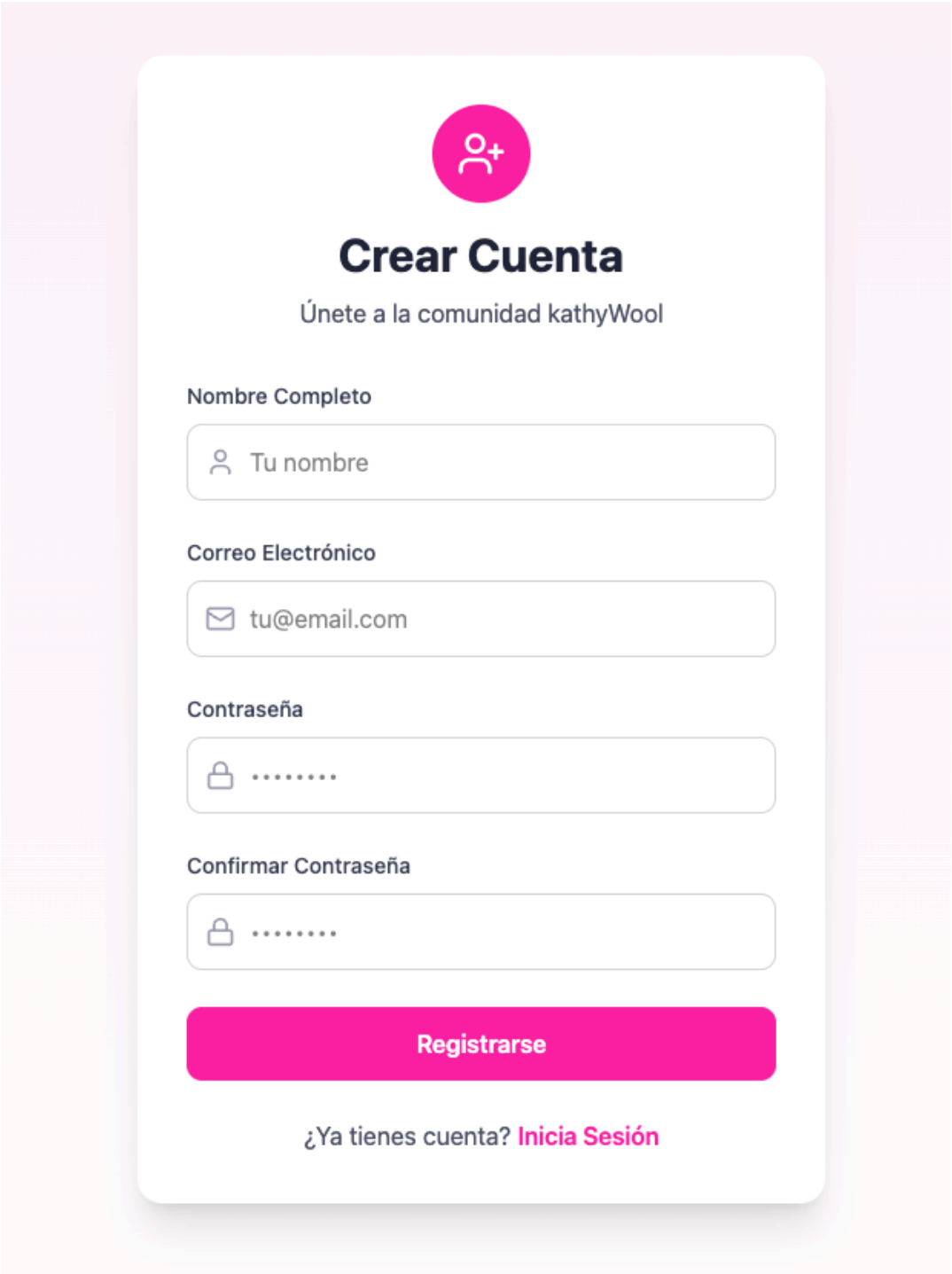
1. Diseño..... 3
2. Esquema de navegación..... 15
3. Dependencias..... 18
4. Diagrama de tablas.....20
5. Contrato.....21

1. Diseño

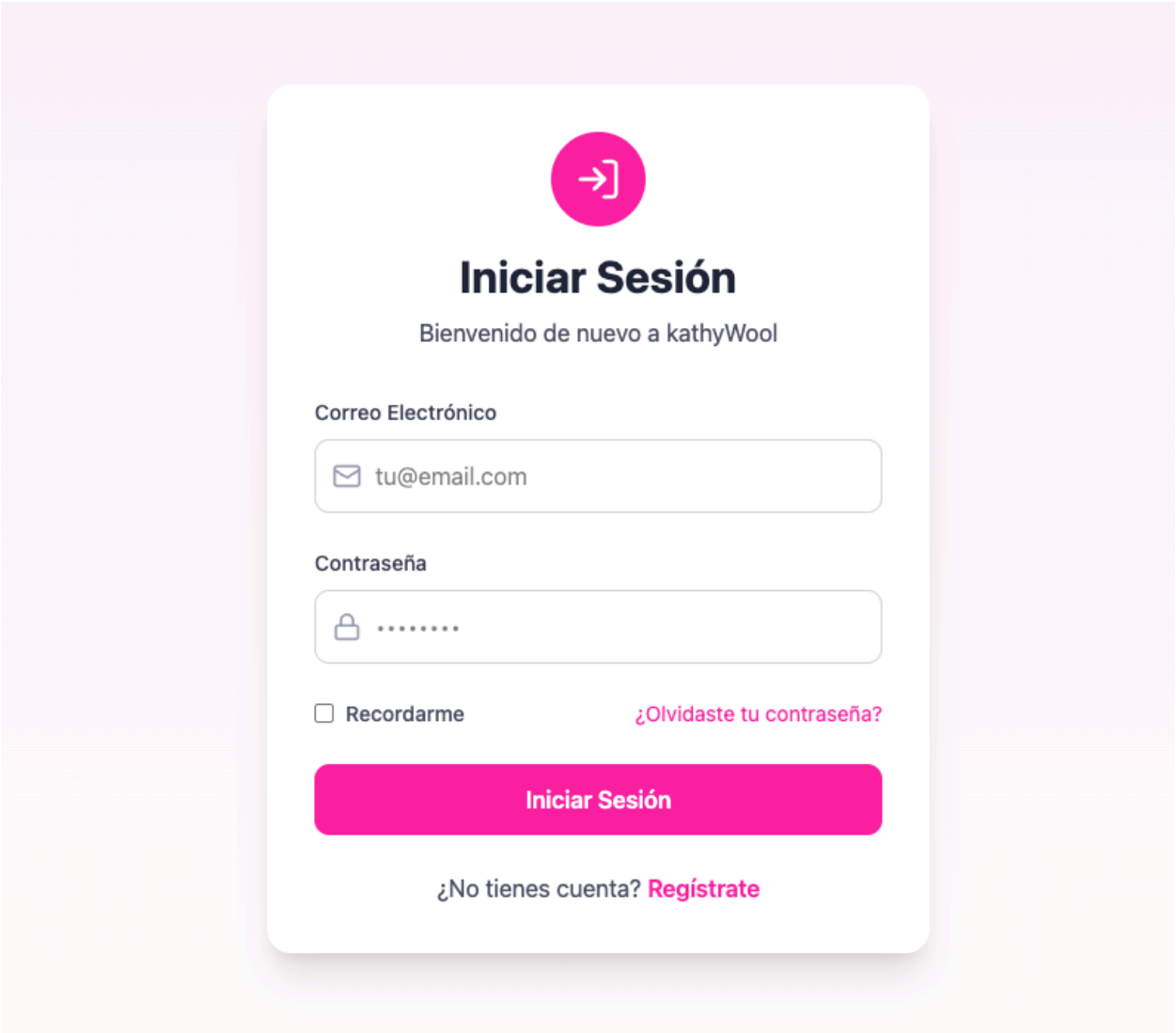
Página Principal



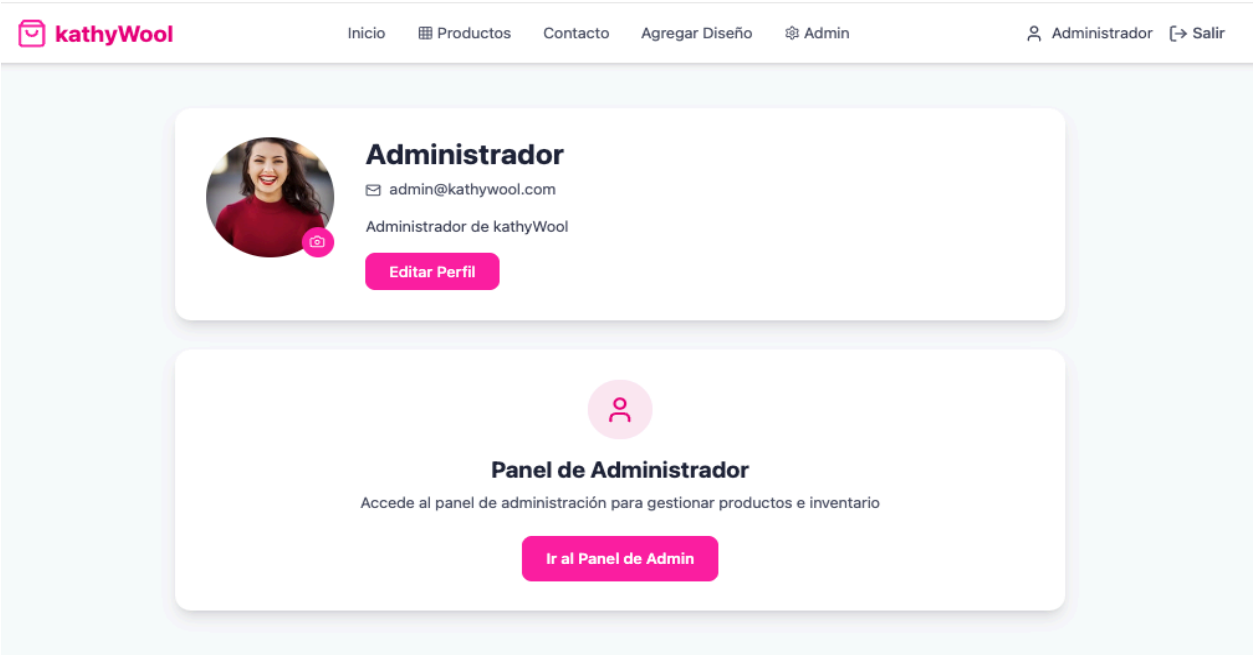
Registro de Usuarios



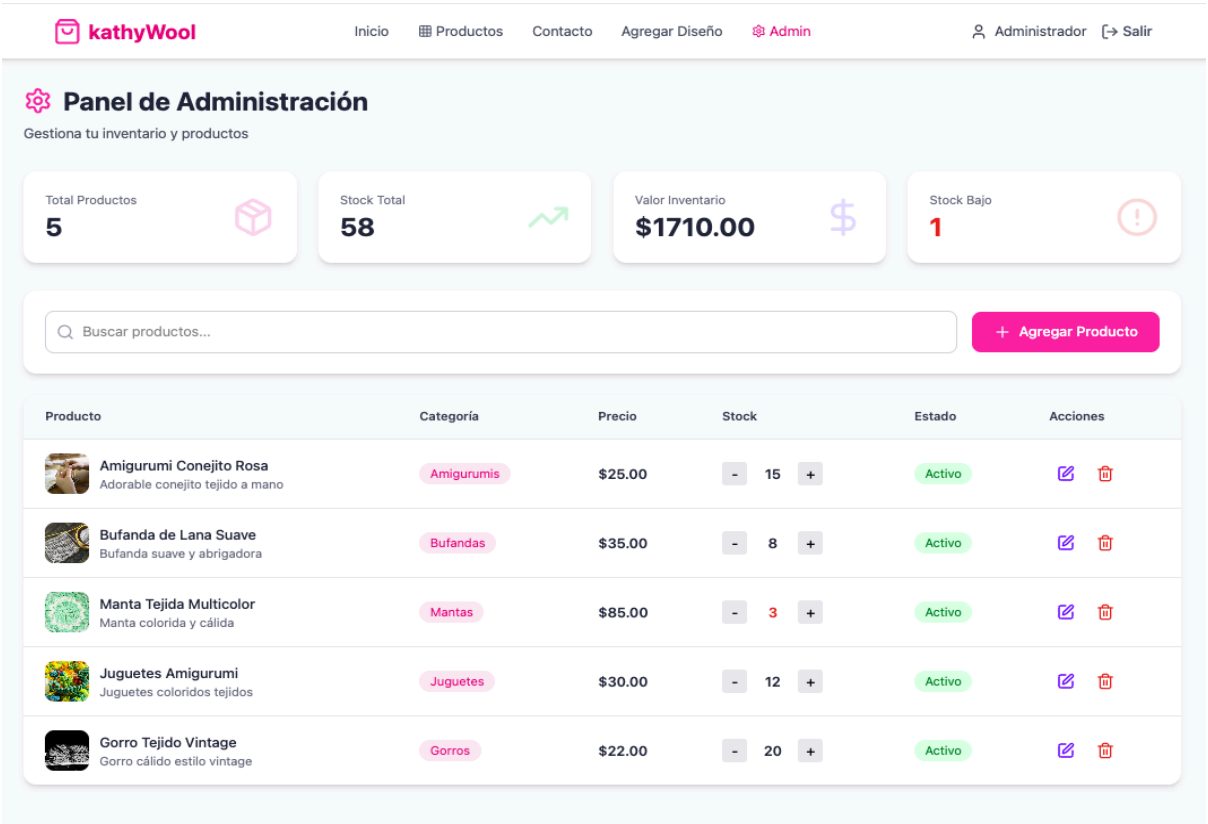
Inicio de Sesión



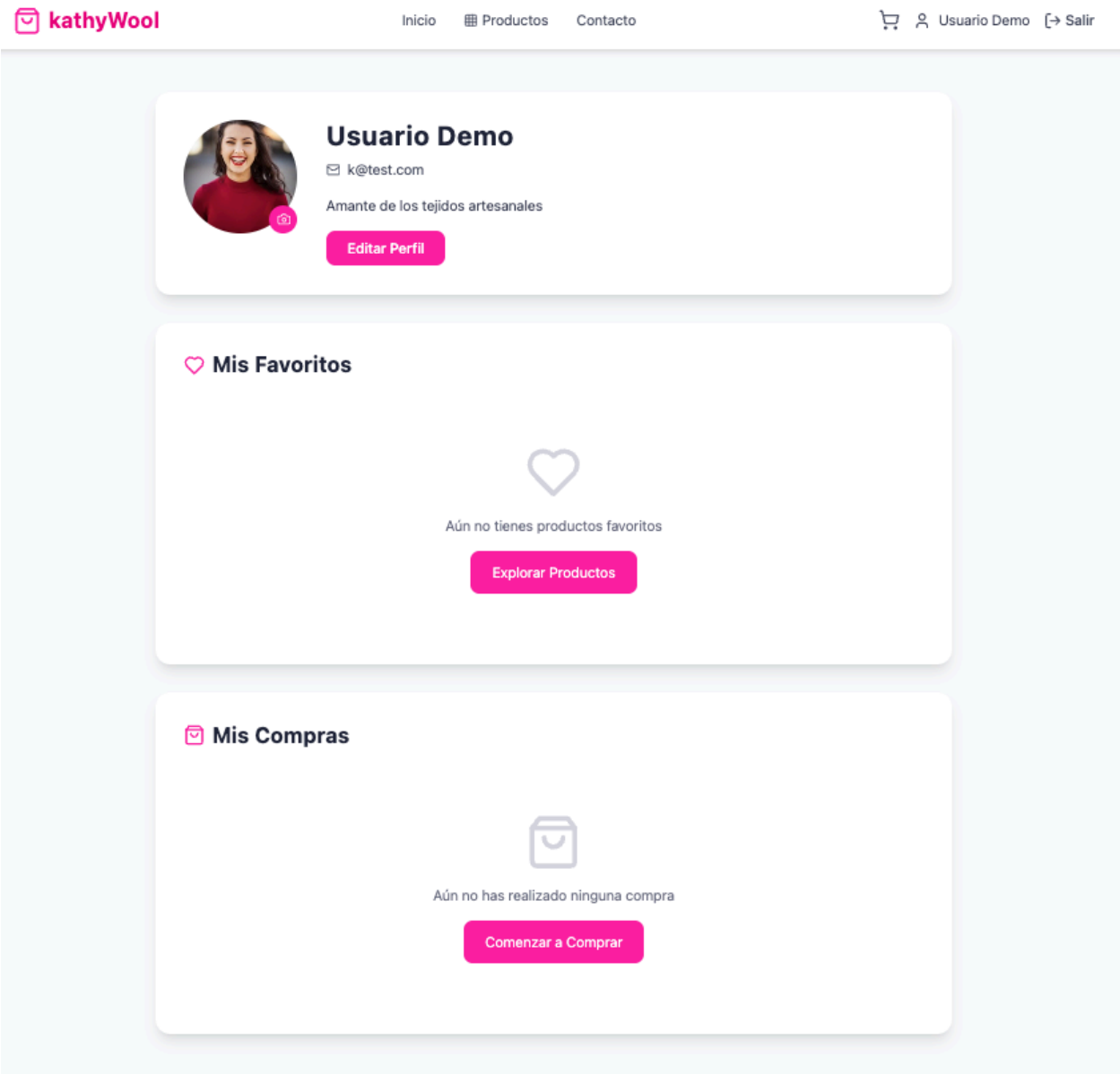
Perfil Administrador



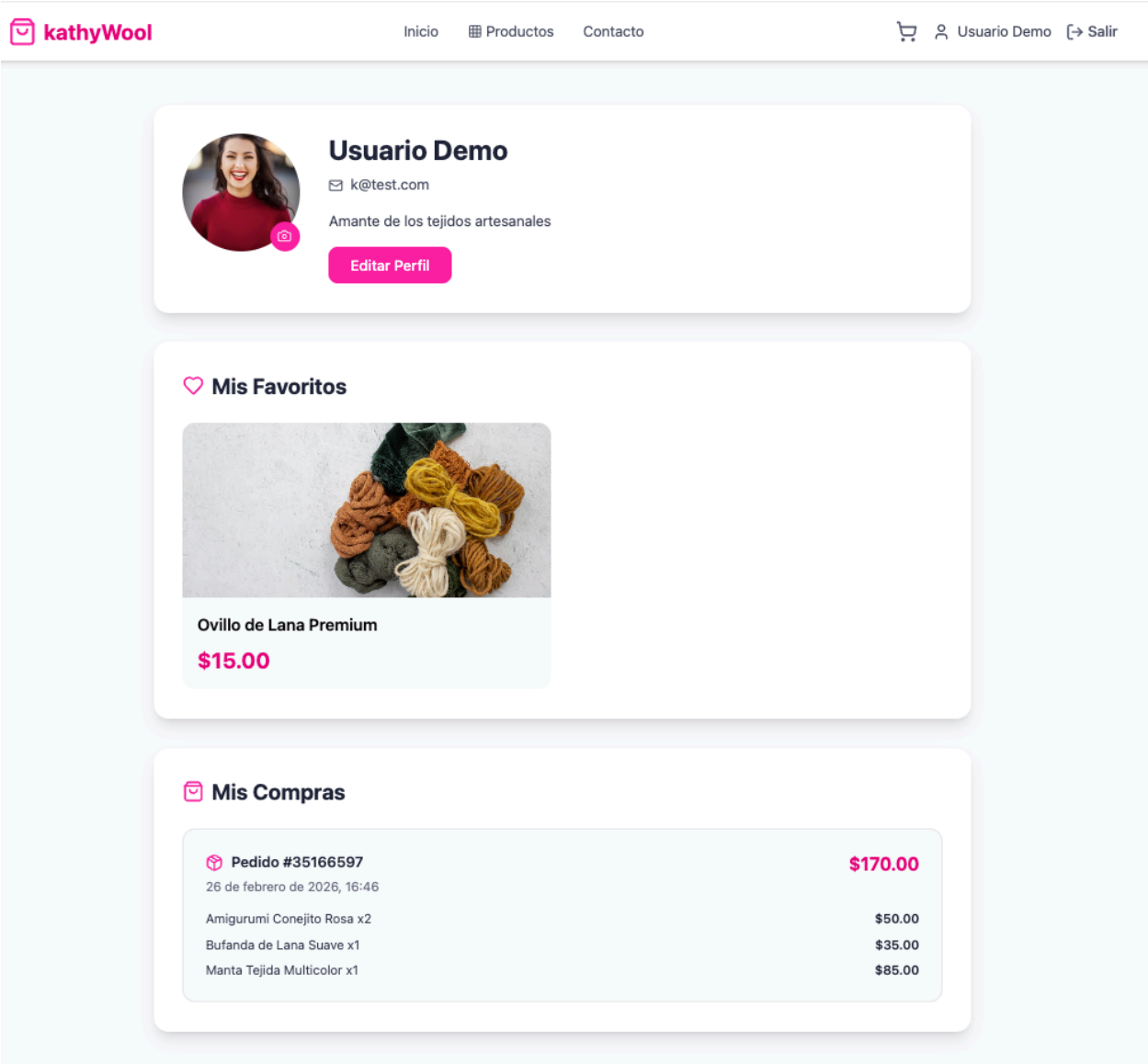
Panel Administrador



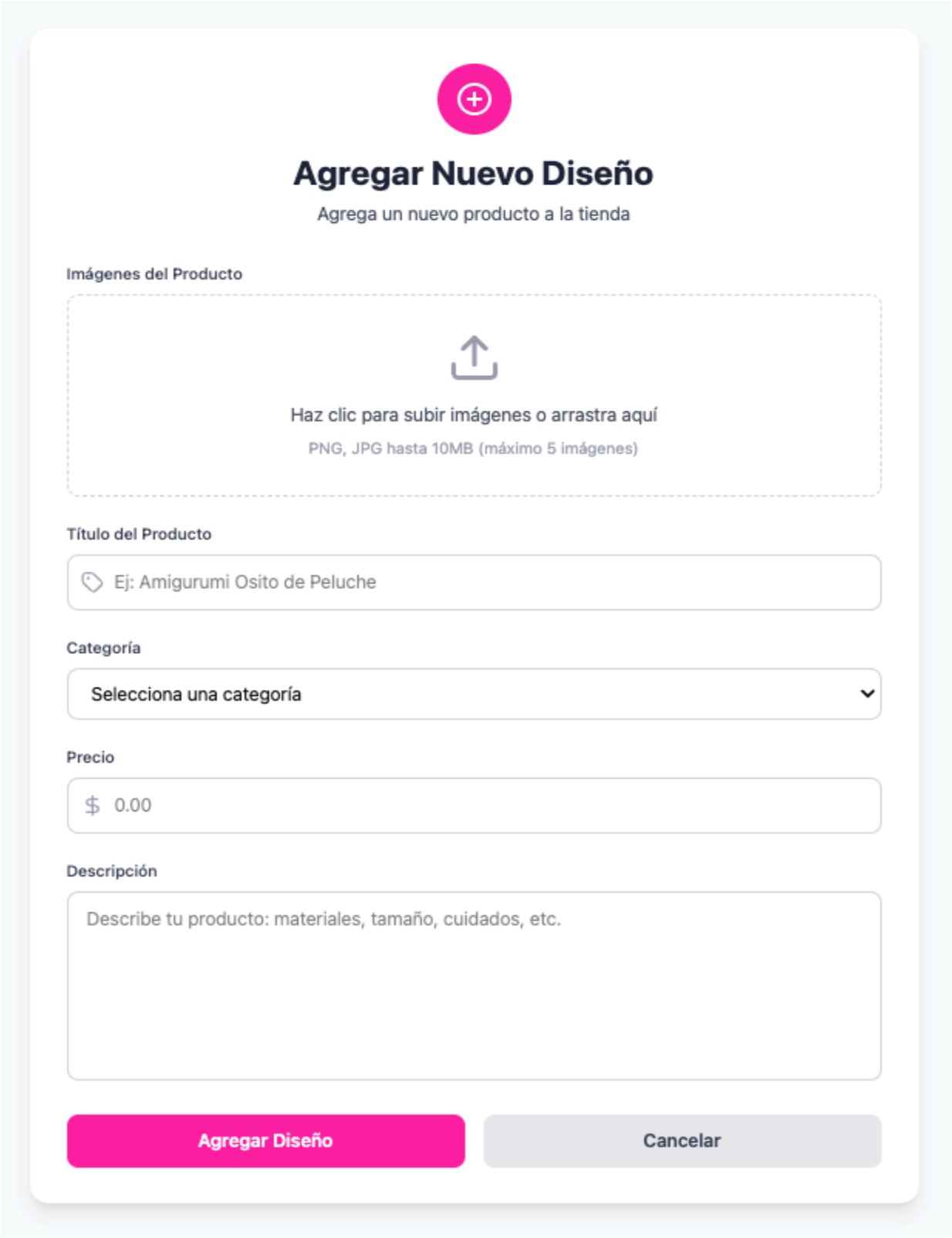
Perfil Usuario



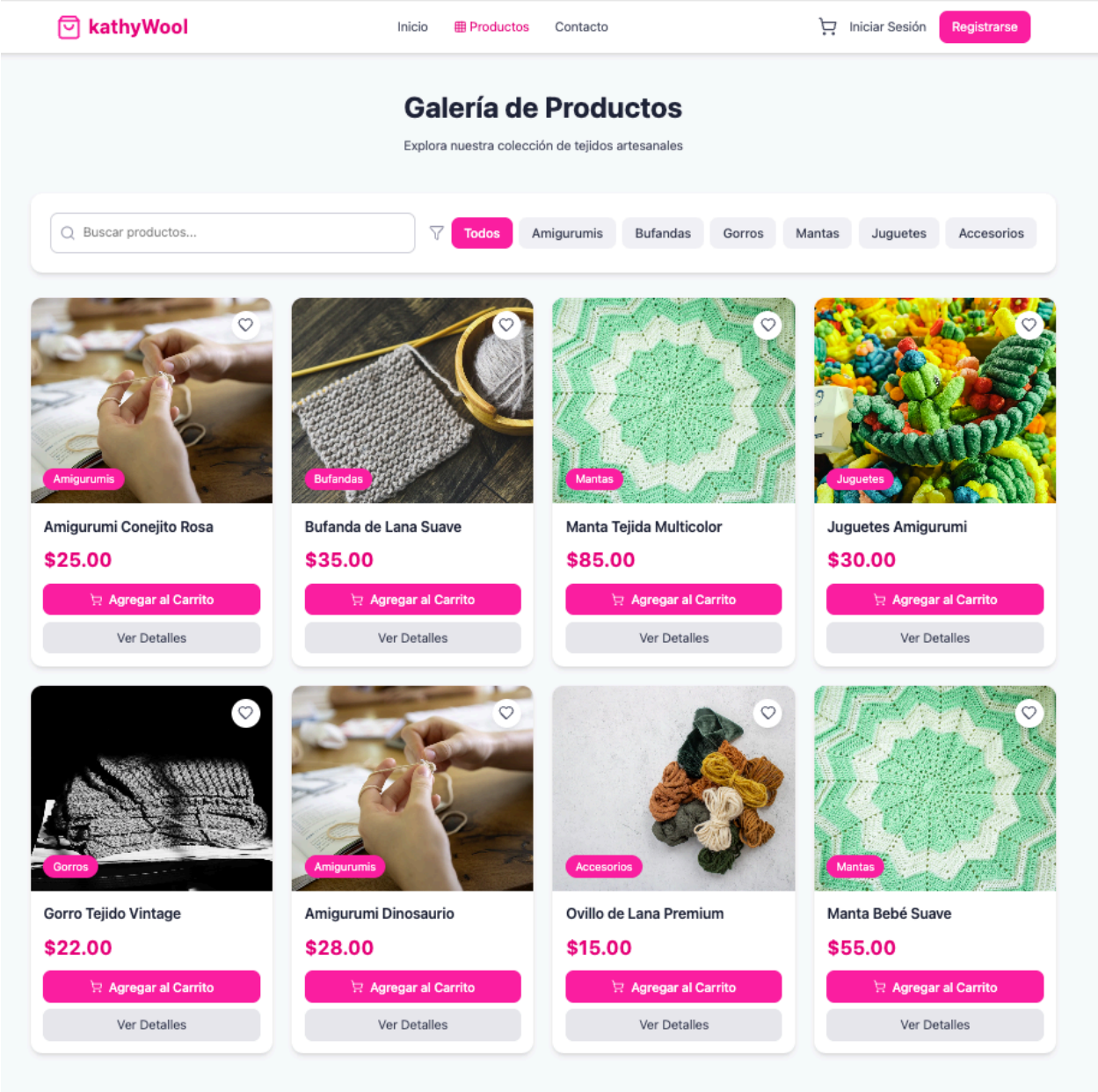
Perfil Usuario 2



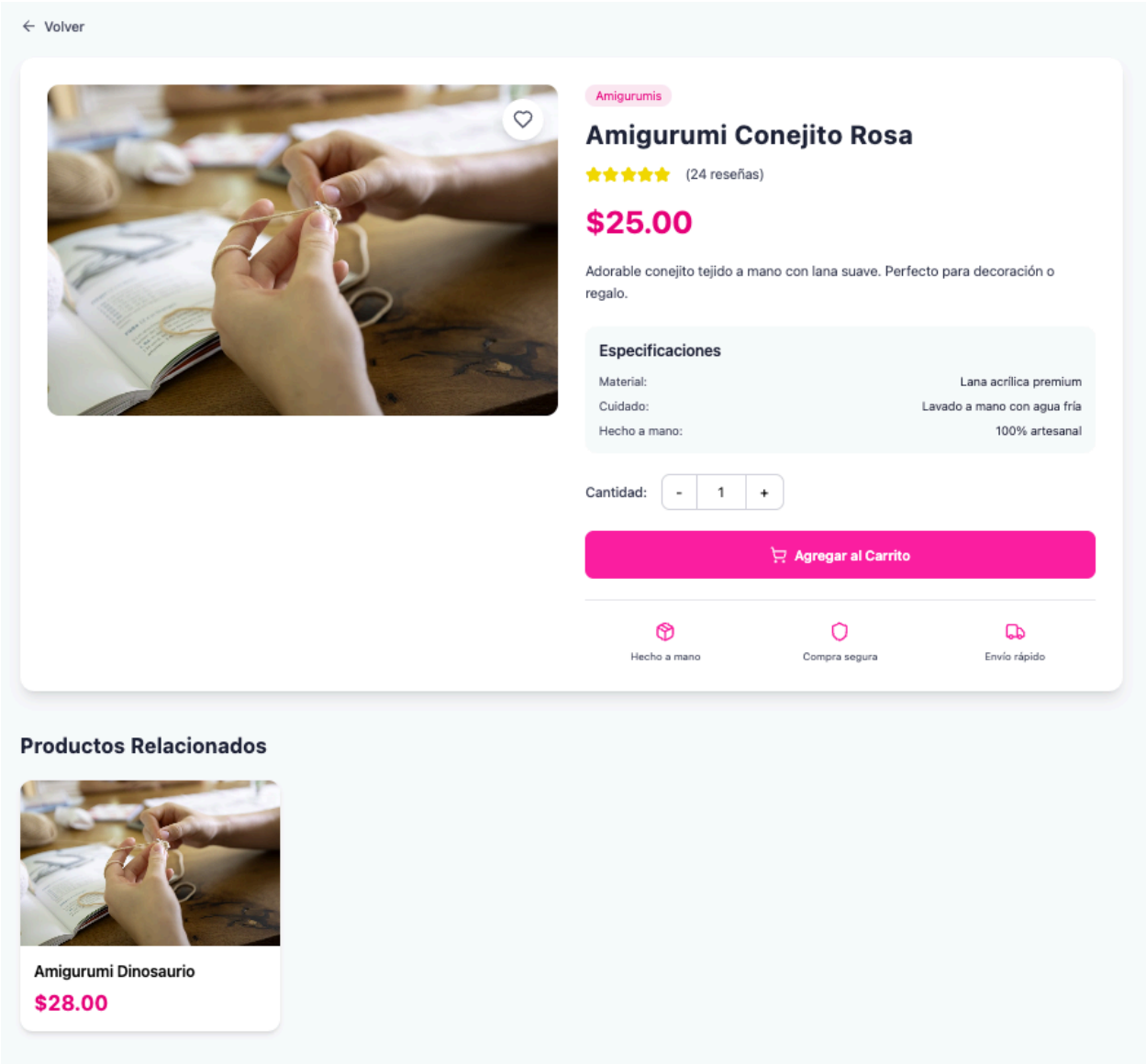
Formulario para crear nuevo Producto



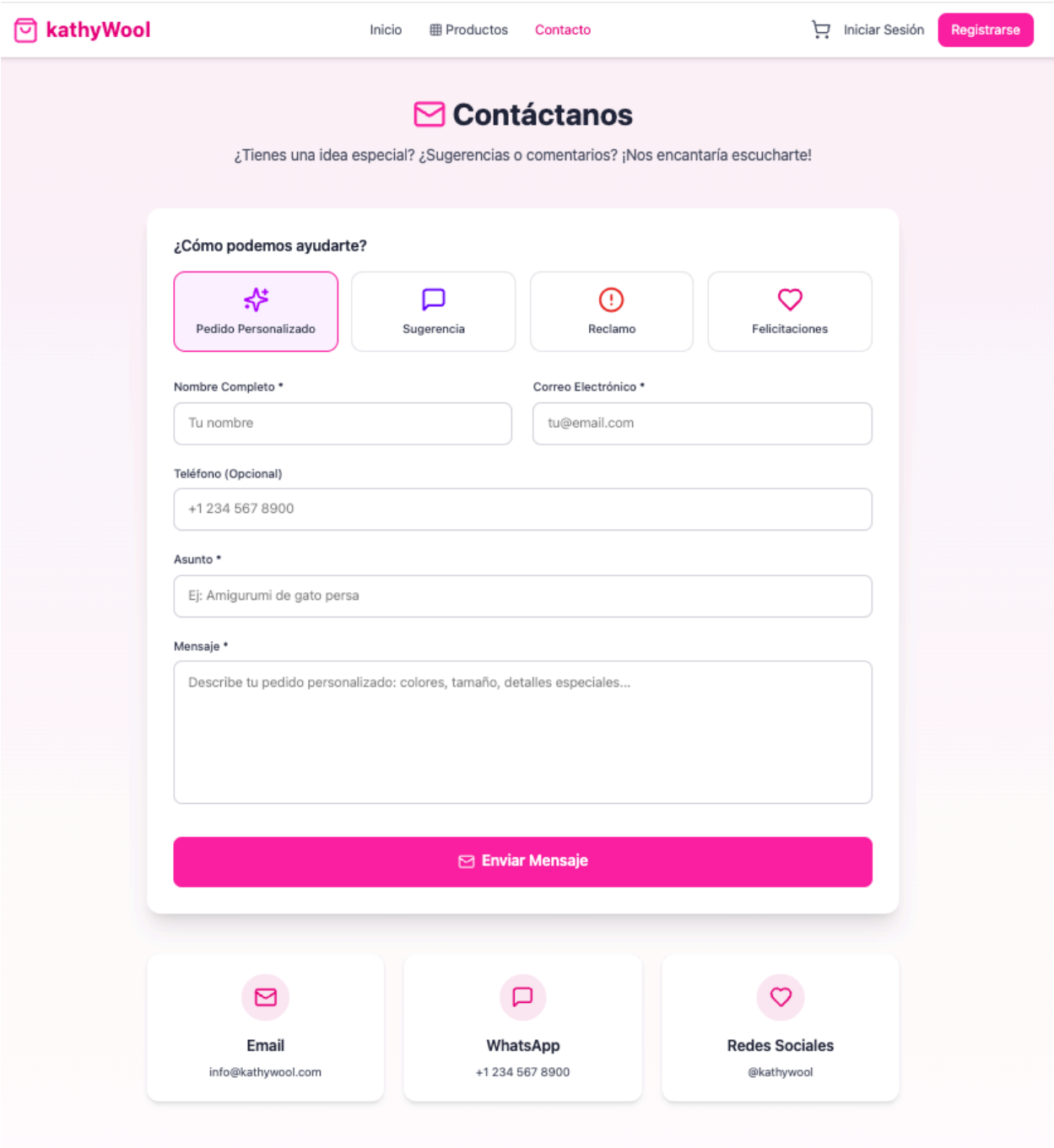
Galería de Productos



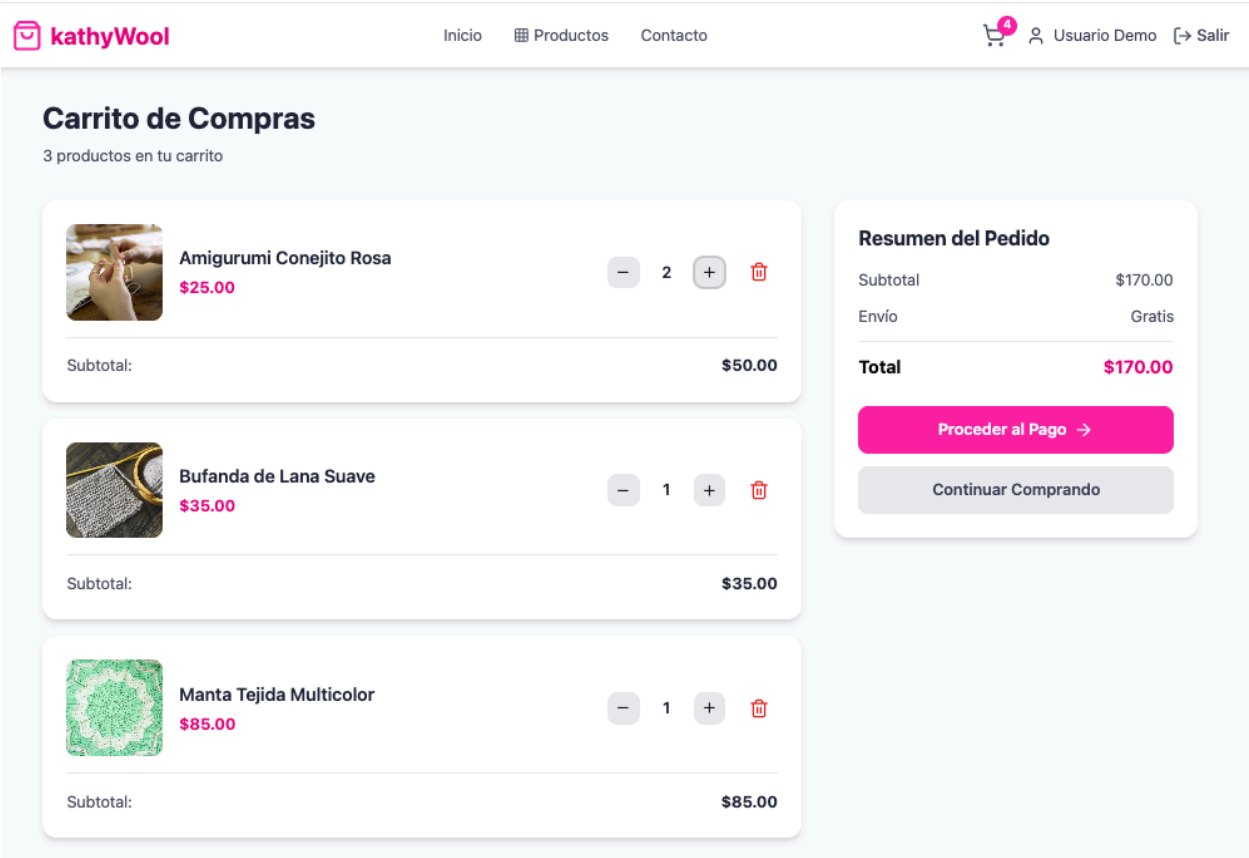
Vista de detalle de un producto



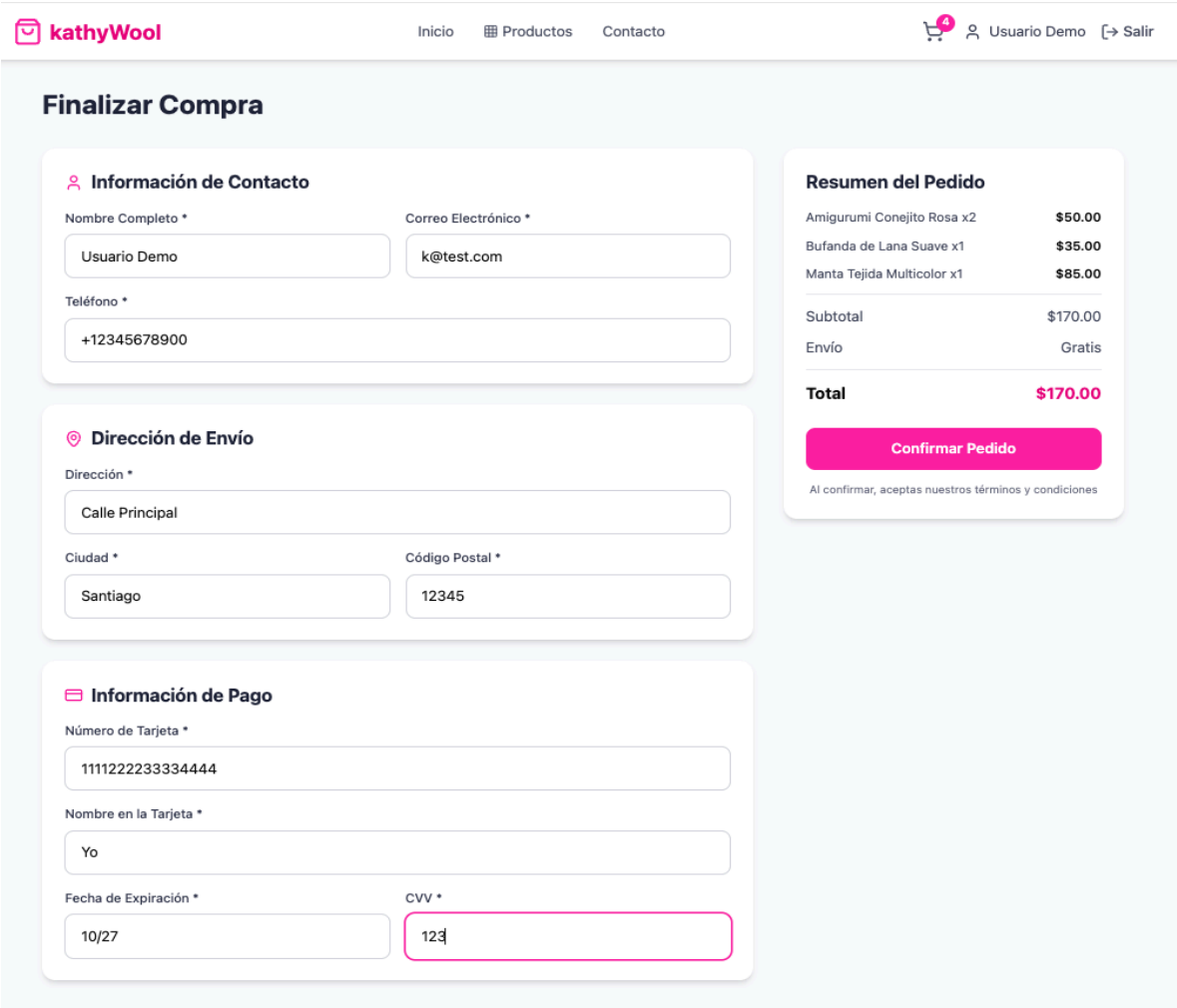
Formulario de Contacto



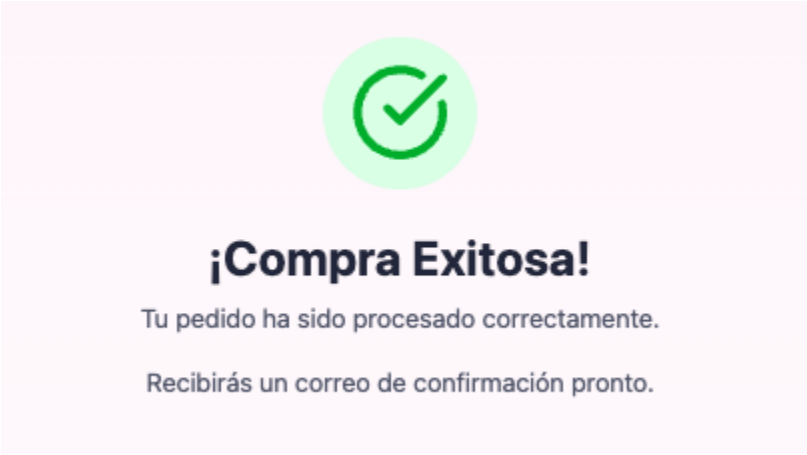
Carrito de Compras



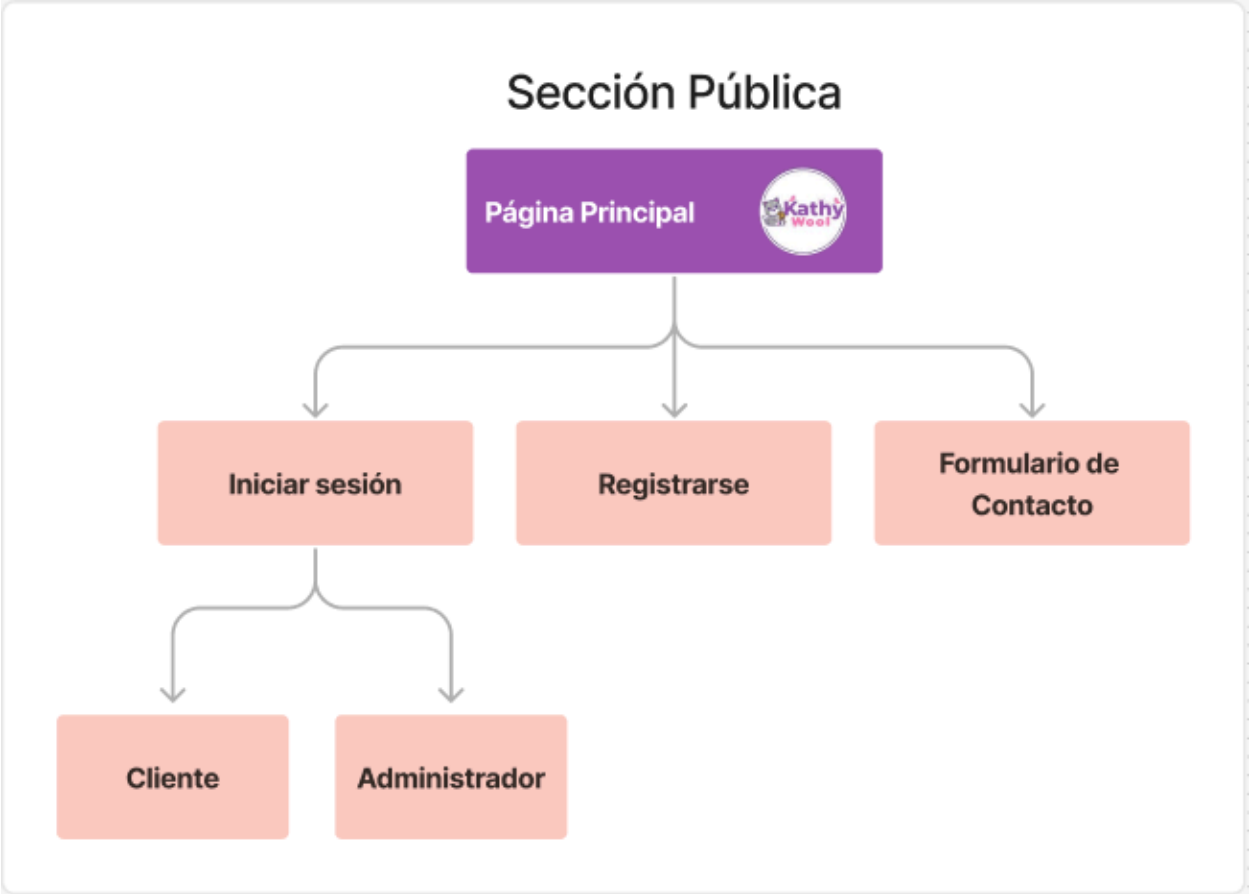
Checkout

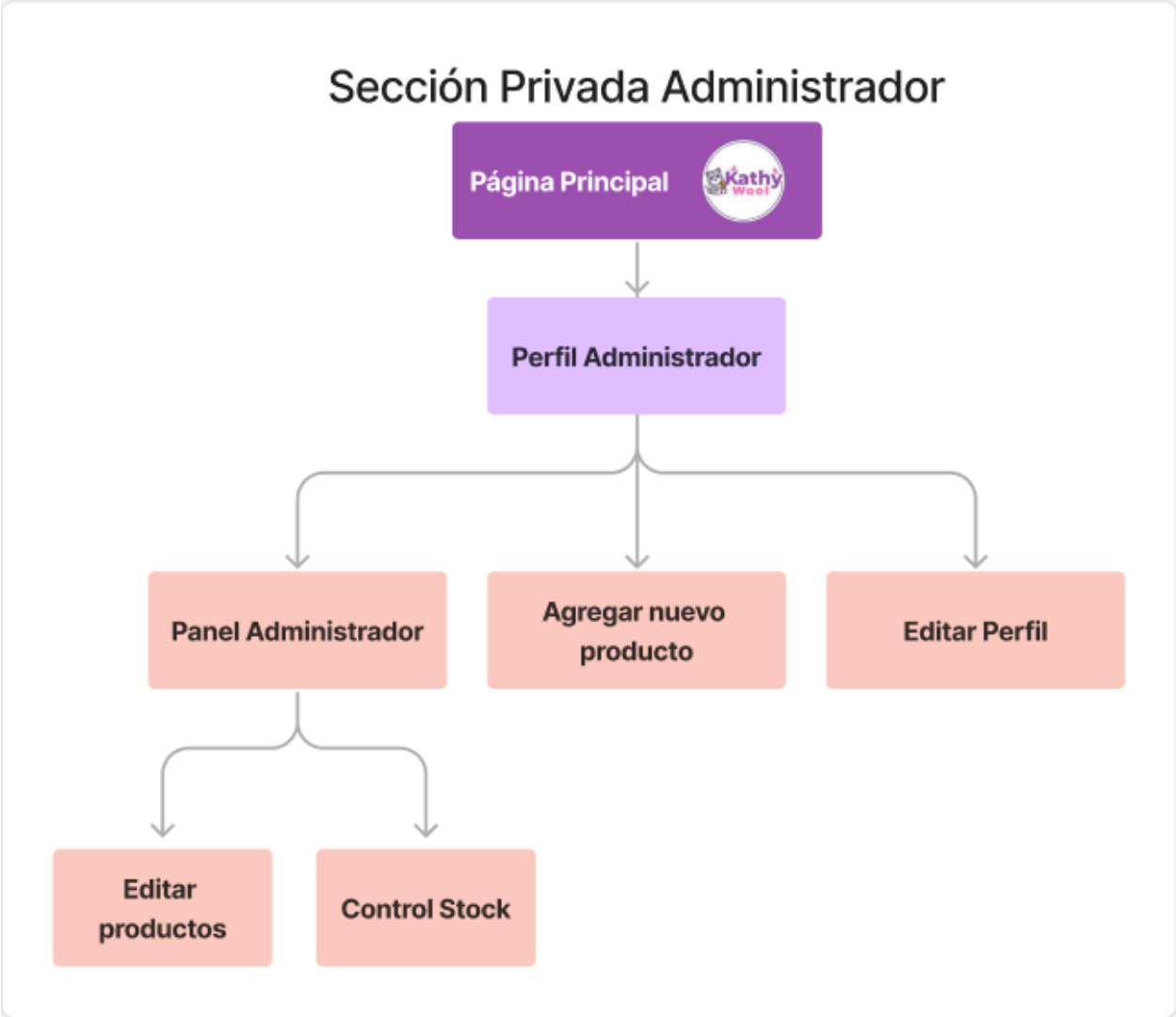


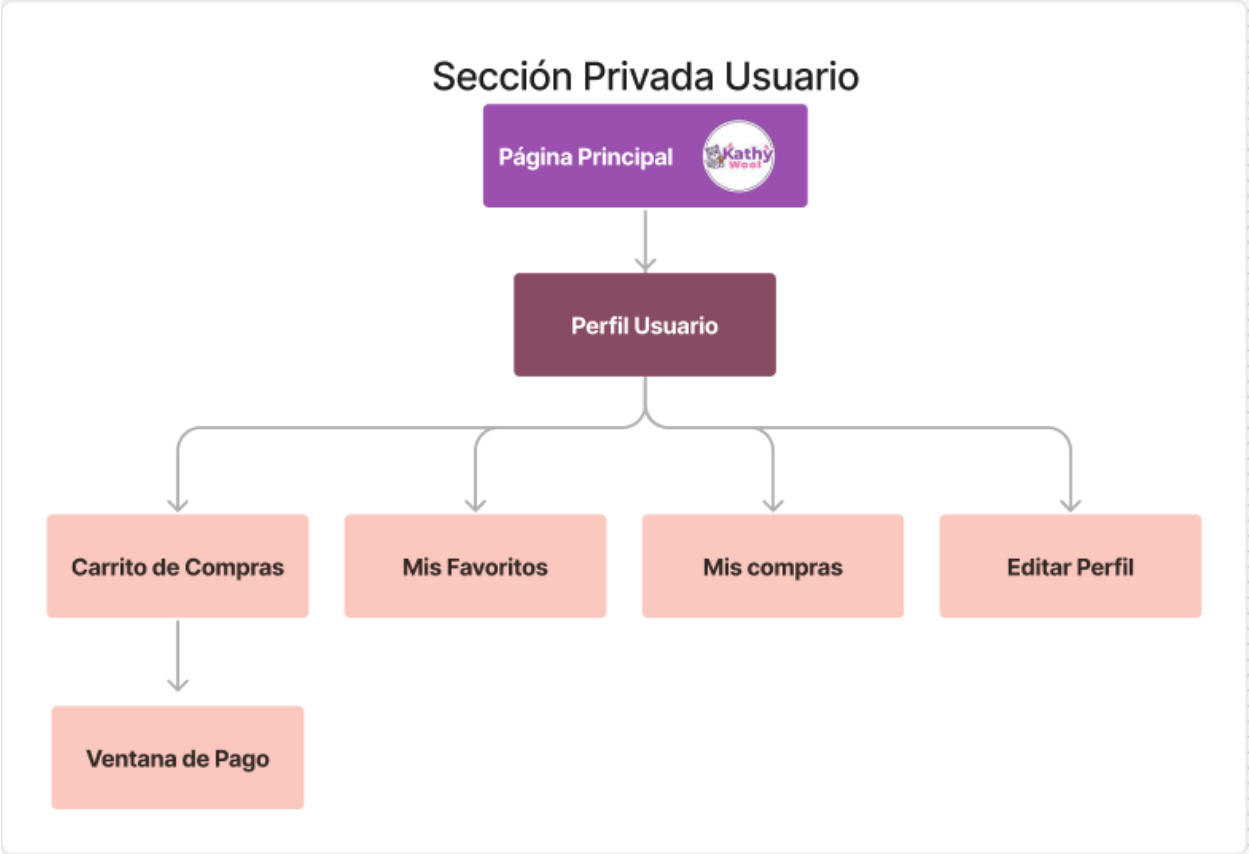
Compra Exitosa



2. Esquema de navegación




Sección Privada Administrador
Página Principal
Kathy Wool
Perfil Administrador
Panel Administrador
Agregar nuevo producto
Editar Perfil
Editar productos
Control Stock


Sección Privada Usuario
Página Principal
Perfil Usuario
Carrito de Compras
Mis Favoritos
Mis compras
Editar Perfil
Ventana de Pago

3. Dependencias

Nuestro proyecto utiliza diversidad de dependencias para facilitar el desarrollo del frontend, backend, conexión con bases de datos y testing. Estas dependencias son gestionadas por medio de npm, que permite tanto su instalación-actualización como el control de sus versiones.

Las dependencias que se estiman requerir para este proyecto, tanto productivas como de desarrollo, se encuentran detalladas en las tablas de la N°1 - 4 (productivas), y la Tabla N°5 (de desarrollo) que se presentan a continuación:

Tabla N° 1: Dependencias de ambiente Productivo para el Frontend		
Dependencia	**Versión**	**Descripción**
React	18.2.0	Librería para construir interfaces de usuario
React DOM	18.2.0	Para renderizar componentes de React en el navegador
Axios	1.6.2	Cliente HTTP para consumir APIs desde el frontend
Bootstrap	5.3.0	Framework de estilos CSS utilizado para diseñar interfaces responsivas y mejorar la presentación visual
@react-fontawesome / @free-solid-svg-icons	0.2.x / 6.5.x	Biblioteca de íconos utilizada en el frontend para mejorar la experiencia visual de la interfaz
Day.js	1.11.18	Librería JavaScript para el manejo de fechas (validar, manipular, y mostrar fechas y horas)
SweetAlert2	11.23.0	Biblioteca de JavaScript para reemplazar ventanas emergentes con modales personalizables, responsivos y accesibles.
React Hook Form	7.x	Biblioteca para gestionar formularios en aplicaciones de React, para manejar estado, validación y rendimiento de formularios

Tabla N° 2: Dependencias de ambiente Productivo para el Backend		
Dependencia	**Versión**	**Descripción**
Express	5.2.1	Framework para la creación de servidores web y manejo de rutas HTTP
Cors	2.8.6	Para habilitar solicitudes entre distintos dominios (CORS)
Dotenv	17.x7	Gestiona variables de entorno desde archivos .env, para proteger credenciales sensibles.
pg	8.18	Cliente para conectarse a bases de datos PostgreSQL

nodemailer	6.9.0	Librería utilizada para gestionar el envío de correos electrónicos como confirmación de registro y compra para el cliente, y también que el administrador pueda recibir nuevos “reclamos, sugerencias, nuevos pedidos, etc”.

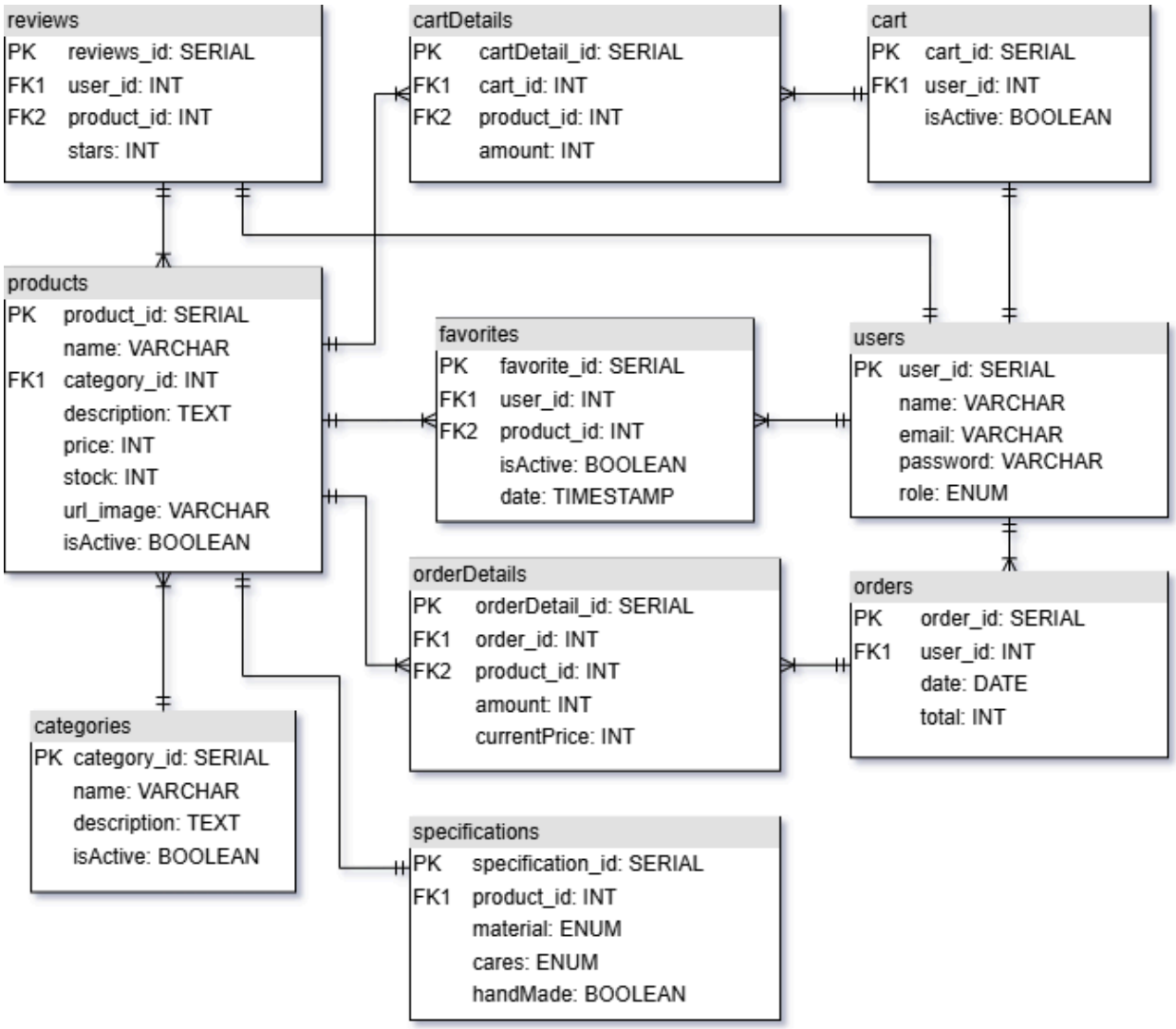
Tabla N° 3: Dependencias de ambiente Productivo para la Seguridad y Autenticación		
Dependencia	**Versión**	**Descripción**
bcrypt	6.0.0	Librería para encriptar contraseñas
jsonwebtoken	9.0.3	Para generar y validar tokens JWT para autenticación

Tabla N° 4: Dependencias de ambiente Productivo para el Testing		
Dependencia	**Versión**	**Descripción**
Supertest	6.3.3	Permite probar endpoints HTTP del servidor

Tabla N° 5: Dependencias de ambiente de Desarrollo		
Dependencia	**Versión**	**Descripción**
Vite	4.4.5	Herramienta para el desarrollo rápido de aplicaciones frontend
@vitejs/plugin-react		Plugin que permite usar React con Vite
EsLint	8.45.0	Analiza el código para mantener buenas prácticas
eslint-plugin-react		Plugin que permite usar React con Vite
eslint-plugin-react-hooks		Reglas de linting específicas para React
eslint-plugin-react-refresh		Validación del uso correcto de hooks
@types/react/ @typer/react-dom		Tipos para trabajar con React (principalmente en entornos tipados)
nodemon	3.x	Permite reiniciar automáticamente el servidor al detectar cambios
Jest	30.2.0	Framework de testing para pruebas unitarias

4. *Diagrama de tablas*

La base de datos contaría con 10 tablas. Las dos tablas principales son **users** y **products**. La tabla **favorites** es una tabla intermedia que destaca contiene cuales son los productos favoritos de los usuarios. La tabla **orders** contiene la información principal de los pedidos realizados por el usuario, mientras que **orderDetails** es una tabla intermedia que reúne el detalle de cuáles y cuántos artículos corresponden a cada pedido. Igualmente, la tabla **cart** contiene los datos principales del carrito de turno de un usuario, mientras que **cartDetails** es una tabla intermedia que reúne el detalle de cuáles y cuántos artículos corresponden a cada carrito. **Categories** es una tabla que contiene todas las categorías agregadas a la app, dándole la posibilidad al usuario de crear nuevas y marcar las antiguas como inactivas. Igualmente, **specifications** contiene las especificaciones de cada artículo, desplegando una serie de alternativas en ENUM para seleccionar. Finalmente, **reviews** es una tabla con las valoraciones que los usuarios hacen de los productos comprados, entre 1 y 5 estrellas.



5. *Contrato*

```yaml
openapi: 3.0.3
info:
  title: Kathy Wool E-commerce API
  version: 2.0.0
  description: Contratos de servicios RESTful (Arquitectura BFF) actualizados
para la plataforma Kathy Wool.
servers:
  - url: /api/v1
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    ErrorResponse:
      type: object
      properties:
        success:
          type: boolean
          example: false
        error:
          type: object
          properties:
            code:
              type: string
              example: "VALIDATION_ERROR"
            message:
              type: string
            details:
              type: array
              items:
                type: string
paths:
  /auth/register:
    post:
      summary: Registro de Nuevo Usuario
```

```yaml
    tags:
      - Autenticación
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [fullName, email, password, passwordConfirmation]
            properties:
              fullName:
                type: string
                minLength: 3
              email:
                type: string
                format: email
              password:
                type: string
                minLength: 8
              passwordConfirmation:
                type: string
    responses:
      '201':
        description: Usuario creado exitosamente

  /auth/login:
    post:
      summary: Inicio de Sesión y Emisión de Credenciales
      tags:
        - Autenticación
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [email, password]
              properties:
                email:
```

```yaml
                type: string
                format: email
              password:
                type: string
              rememberMe:
                type: boolean
    responses:
      '200':
        description: Autenticación exitosa
        content:
          application/json:
            schema:
              type: object
              properties:
                success:
                  type: boolean
                  example: true
                data:
                  type: object
                  properties:
                    token:
                      type: string
                    expiresIn:
                      type: integer
                    user:
                      type: object
                      properties:
                        id:
                          type: integer
                        role:
                          type: string

  /products:
    get:
      summary: Listado Compuesto y Consulta de Catálogo
      tags:
        - Catálogo
      parameters:
        - name: searchKeyword
```

```yaml
        in: query
        schema:
          type: string
      - name: categoryFilter
        in: query
        schema:
          type: string
      - name: page
        in: query
        schema:
          type: integer
          default: 1
    responses:
      '200':
        description: Colección paginada de productos
        content:
          application/json:
            schema:
              type: object
              properties:
                success:
                  type: boolean
                  example: true
                data:
                  type: array
                  items:
                    type: object
                    properties:
                      id:
                        type: integer
                      name:
                        type: string
                      category:
                        type: string
                      priceDetails:
                        type: object
                      stockAvailable:
                        type: integer
```

```yaml
/contact:
  post:
    summary: Enviar mensaje de contacto
    tags:
      - Comunicación
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [fullName, email, subject, message]
            properties:
              fullName:
                type: string
              phone:
                type: string
              email:
                type: string
                format: email
              subject:
                type: string
              message:
                type: string
    responses:
      '200':
        description: Mensaje recibido por la plataforma

/users/me/favorites:
  post:
    summary: Añadir artículo a Favoritos (Lista de Deseos)
    tags:
      - Interacción de Usuario
    security:
      - bearerAuth:
    requestBody:
      required: true
      content:
        application/json:
```

```yaml
          schema:
            type: object
            required: [productId]
            properties:
              productId:
                type: integer
    responses:
      '201':
        description: Producto añadido a favoritos

  /users/me/orders:
    get:
      summary: Historial de compras del usuario (Mis Compras)
      tags:
        - Interacción de Usuario
      security:
        - bearerAuth:
      responses:
        '200':
          description: Lista de pedidos realizados por el usuario
          content:
            application/json:
              schema:
                type: object
                properties:
                  success:
                    type: boolean
                  data:
                    type: array
                    items:
                      type: object
                      properties:
                        orderId:
                          type: string
                        date:
                          type: string
                          format: date-time
                        items:
                          type: array
```

```yaml
                        items:
                          type: object
                          properties:
                            name:
                              type: string
                            quantity:
                              type: integer
                            price:
                              type: number
                      total:
                        type: number

  /users/me/favorites/{productId}:
    delete:
      summary: Remover artículo de Favoritos
      tags:
        - Interacción de Usuario
      security:
        - bearerAuth:
      parameters:
        - name: productId
          in: path
          required: true
          schema:
            type: integer
      responses:
        '204':
          description: Eliminación lógica exitosa sin contenido de retorno

  /admin/metrics:
    get:
      summary: Obtener métricas del panel de administración
      tags:
        - Administración
      security:
        - bearerAuth:
      responses:
        '200':
          description: Resumen de inventario y estado global
```

```yaml
        content:
          application/json:
            schema:
              type: object
              properties:
                success:
                  type: boolean
                data:
                  type: object
                  properties:
                    totalProducts:
                      type: integer
                    totalStock:
                      type: integer
                    inventoryValue:
                      type: number
                      format: float
                    lowStockCount:
                      type: integer

/orders/checkout:
  post:
    summary: Consolidación y Finalización de Pedido
    tags:
      - Transaccional
    security:
      - bearerAuth:
    parameters:
      - name: Idempotency-Key
        in: header
        required: true
        schema:
          type: string
          format: uuid
    requestBody:
      required: true
      content:
        application/json:
          schema:
```

```yaml
            type: object
            required: [cartId, contactInfo, shippingAddress, paymentInfo]
            properties:
              cartId:
                type: string
              contactInfo:
                type: object
                required: [fullName, email]
                properties:
                  fullName:
                    type: string
                  phone:
                    type: string
                  email:
                    type: string
                    format: email
              shippingAddress:
                type: object
                required: [address, city, zipCode]
                properties:
                  address:
                    type: string
                  city:
                    type: string
                  zipCode:
                    type: string
              paymentInfo:
                type: object
                required: [cardNumber, cardName, expiry, cvv]
                properties:
                  cardNumber:
                    type: string
                  cardName:
                    type: string
                  expiry:
                    type: string
                  cvv:
                    type: string
    responses:
```

```yaml
        '201':
          description: TransacciÃ³n completada exitosamente
        '422':
          description: Fallo en validación (ej. inventario insuficiente)
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ErrorResponse'
```